

Data-Parallel Training

July 29: Gradient Averaging from First Principles

Model replicas • data shards • all-reduce • manual simulation • DDP preview

DP

Big idea

Replicas train on different data, then synchronize updates.

▽

Student output

Implement simulated data-parallel training with weight averaging.

Today's Roadmap

From MPI collectives to distributed training

01

Recap

broadcast/scatter/
reduce/all-reduce

02

Data parallelism

replicas, shards,
batches

03

Gradient averaging

local gradient →
global update

04

DDP preview

real PyTorch
distributed
training

05

Lab

simulate
distributed
training

1. Recap and Motivation

Day 5 collectives become Day 6 training synchronization

Day 5 Recap: Collectives for Training

Distributed training is built from the same communication patterns

B

Broadcast

send initial model parameters to all workers

S

Scatter

give each worker a different data shard

Σ

All-reduce

average gradients so all replicas update consistently

Today: we use these collectives to train synchronized model replicas

Why Distributed Training?

Training cost grows with data, models, and experiments



More data

split examples across workers



Faster epochs

process larger effective batches



Larger experiments

run scaling and ablations



Real AI systems

multi-GPU/multi-node training



Tradeoffs

communication can bottleneck



Reproducibility

seeds, logs, checkpoints matter

2. Data-Parallel Mental Model

Same model, different data, synchronized updates

Data Parallelism vs Model Parallelism

Two different ways to distribute training

DP

Data parallelism

Every worker has a full model replica. Each worker trains on different data. Gradients are averaged before updates.

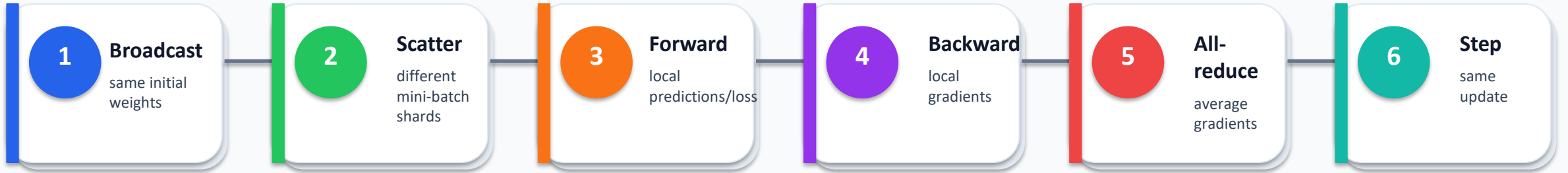
MP

Model parallelism

Different parts of a model live on different devices. Useful when one model is too large for one device.

Data-Parallel Training Loop

Replicas must stay synchronized



If all replicas start equal and use averaged gradients, they remain equal

Effective Batch Size

More workers often means a larger total batch

B

Per-worker batch

number of samples each
replica processes locally

N

World size

number of participating
workers or replicas

G

Global batch

per-worker batch $\times$ world size

Changing global batch may require learning-rate tuning

3. Gradient Averaging from First Principles

The math behind all-reduce training

Single-Worker Gradient Descent

Baseline training update

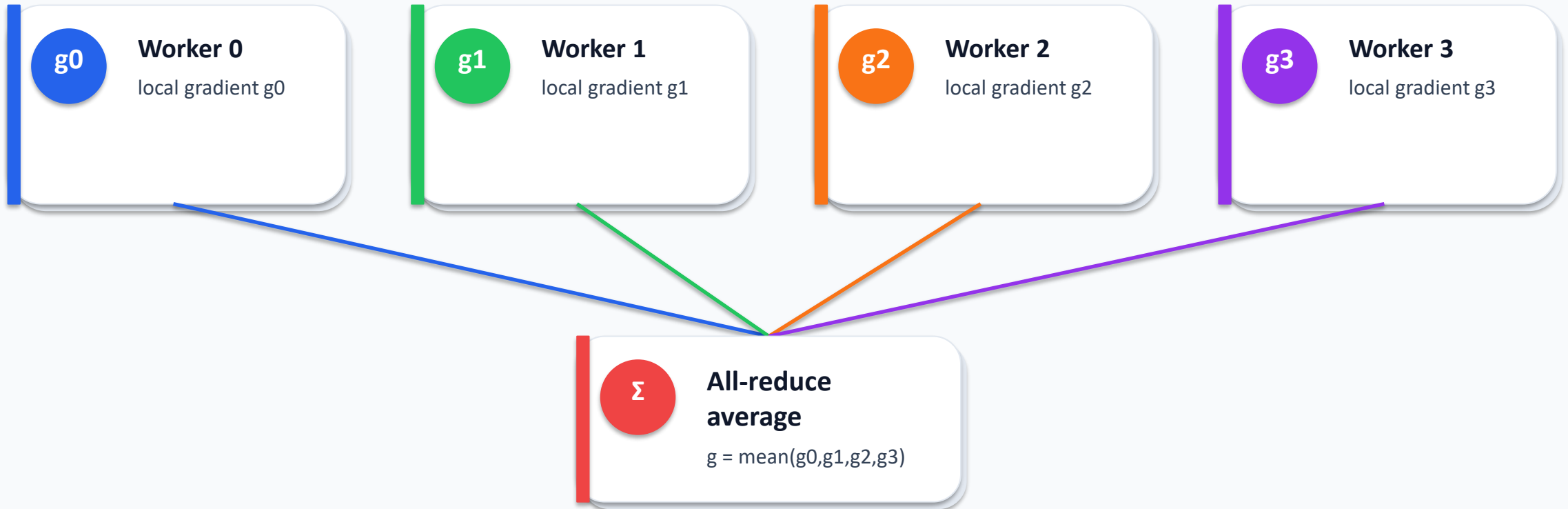


```
# Single-worker SGD idea
prediction = model(x)
loss = loss_fn(prediction, y)
gradients = compute_gradients(loss, model)

for parameter, gradient in zip(model.parameters(), gradients):
    parameter = parameter - learning_rate * gradient
```

Data-Parallel Gradient Averaging

Each worker contributes a local gradient



Every worker receives the same averaged gradient and applies the same update

Weight Averaging vs Gradient Averaging

Two useful simulations, but not exactly the same

▽

Gradient averaging

Synchronize every step: average gradients, then update all replicas. This is closest to DDP.

W

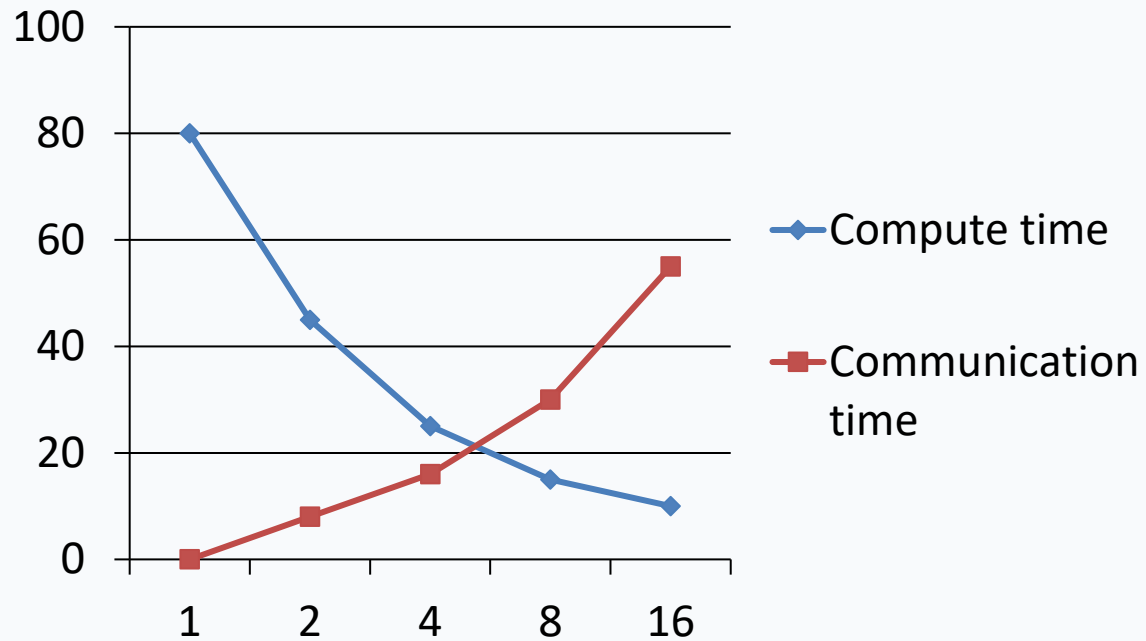
Weight averaging

Workers train locally for some steps, then average weights. Easier to simulate; resembles federated averaging.

Communication Cost

All-reduce can become the bottleneck

Example compute vs communication tradeoff



More workers

less local compute per worker



More communication

more synchronization overhead



Sweet spot

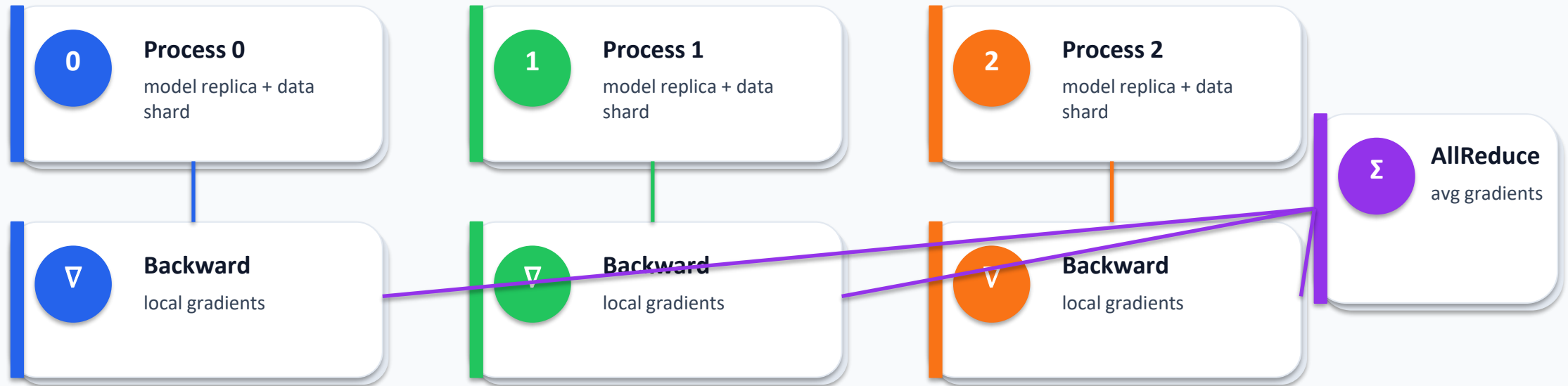
best wall-clock time occurs before communication dominates

4. DDP Preview

How PyTorch DistributedDataParallel automates synchronization

PyTorch DDP Mental Model

What real frameworks automate



What DDP Handles Automatically

Useful concepts even if we simulate manually today

1

Process group

communication context among ranks

2

Initial sync

broadcast model state from rank 0

3

Gradient sync

all-reduce during backward pass

4

Buckets

group gradients for efficient communication

5

DistributedSampler

each rank sees different samples

6

Launchers

torchrun or mp.spawn start processes

Common Distributed Training Pitfalls

Real systems fail in predictable ways

1

Different seeds

replicas diverge unexpectedly

2

Wrong sampler

ranks train on duplicate data

3

Mismatched shapes

collective operations fail

4

No eval-only rank logic

metrics double-count data

5

Too frequent sync

communication dominates

6

Poor logging

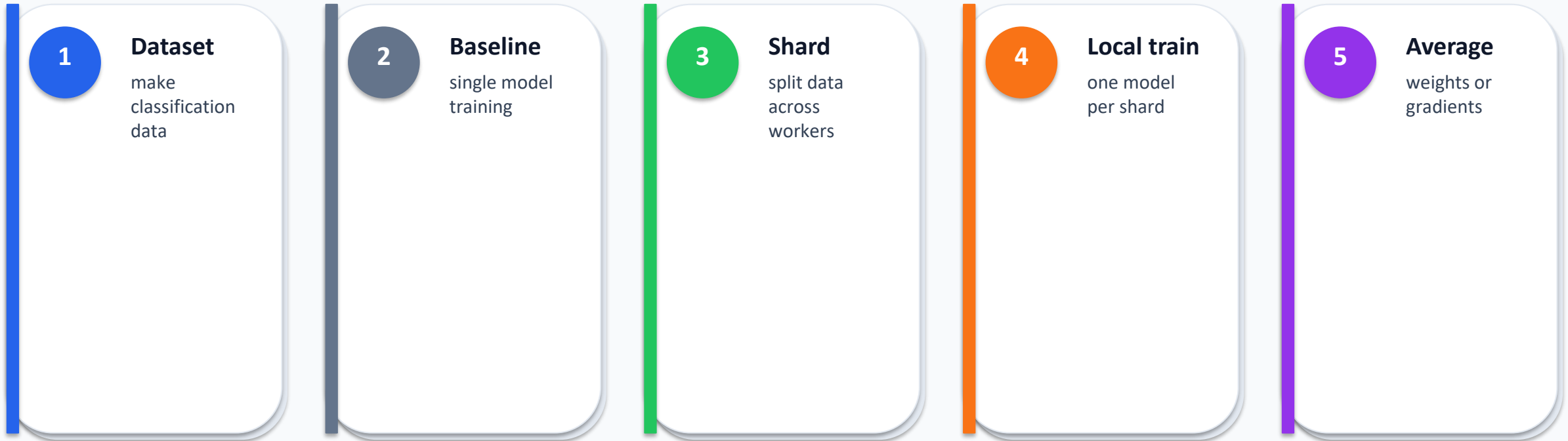
hard to debug which rank failed

5. Day 6 Lab

Simulate data-parallel training with local models

Lab: Distributed Training Simulation

Goal: train local models on shards and synchronize updates



Deliverable: loss/accuracy curves + sync strategy explanation + benchmark table

Lab Starter: Dataset and Model

Use NumPy logistic regression for transparent math

```
import numpy as np

def make_data(n=4000, d=20, seed=42):
    rng = np.random.default_rng(seed)
    X = rng.normal(size=(n, d))
    true_w = rng.normal(size=d)
    logits = X @ true_w
    y = (logits > 0).astype(np.float64)
    return X, y

def sigmoid(z):
    return 1 / (1 + np.exp(-z))
```

Serial Baseline Training

Train one model on all data

```
def loss_and_grad(w, X, y):  
    p = sigmoid(X @ w)  
    eps = 1e-8  
    loss = -np.mean(y*np.log(p+eps) + (1-y)*np.log(1-p+eps))  
    grad = X.T @ (p - y) / len(y)  
    return loss, grad  
  
def train_serial(X, y, lr=0.5, epochs=50):  
    w = np.zeros(X.shape[1])  
    losses = []  
    for _ in range(epochs):  
        loss, grad = loss_and_grad(w, X, y)  
        w -= lr * grad  
        losses.append(loss)  
    return w, losses
```

Simulated Data-Parallel Step

Average local gradients before updating

```
def shard_data(X, y, world_size):  
    return [(X[i::world_size], y[i::world_size]) for i in range(world_size)]  
  
def train_data_parallel(X, y, world_size=4, lr=0.5, epochs=50):  
    shards = shard_data(X, y, world_size)  
    w = np.zeros(X.shape[1])  
    losses = []  
  
    for _ in range(epochs):  
        local_grads, local_losses = [], []  
        for Xs, ys in shards:  
            loss, grad = loss_and_grad(w, Xs, ys)  
            local_losses.append(loss)  
            local_grads.append(grad)  
        avg_grad = np.mean(local_grads, axis=0) # all-reduce average  
        w -= lr * avg_grad  
        losses.append(float(np.mean(local_losses)))  
    return w, losses
```

Evaluation and Correctness

Compare accuracy, loss curves, and final weights

```
def accuracy(w, X, y):  
    pred = (sigmoid(X @ w) >= 0.5).astype(float)  
    return np.mean(pred == y)  
  
X, y = make_data()  
serial_w, serial_losses = train_serial(X, y)  
dp_w, dp_losses = train_data_parallel(X, y, world_size=4)  
  
print("serial acc:", accuracy(serial_w, X, y))  
print("data-parallel acc:", accuracy(dp_w, X, y))  
print("weight distance:", np.linalg.norm(serial_w - dp_w))
```

Weight Averaging / Federated Style

Local training then average model weights

```
def local_train(w, X, y, lr=0.5, local_steps=5):
    w = w.copy()
    for _ in range(local_steps):
        _, grad = loss_and_grad(w, X, y)
        w -= lr * grad
    return w

def train_weight_averaging(X, y, world_size=4, rounds=20):
    shards = shard_data(X, y, world_size)
    w = np.zeros(X.shape[1])
    for _ in range(rounds):
        local_weights = [local_train(w, Xs, ys) for Xs, ys in shards]
        w = np.mean(local_weights, axis=0)
    return w
```

Mini Challenge

Compare synchronization strategies

1

Run

serial, gradient-averaging,
and weight-averaging
training.

2

Record

loss curve, accuracy,
runtime, sync frequency,
world size.

3

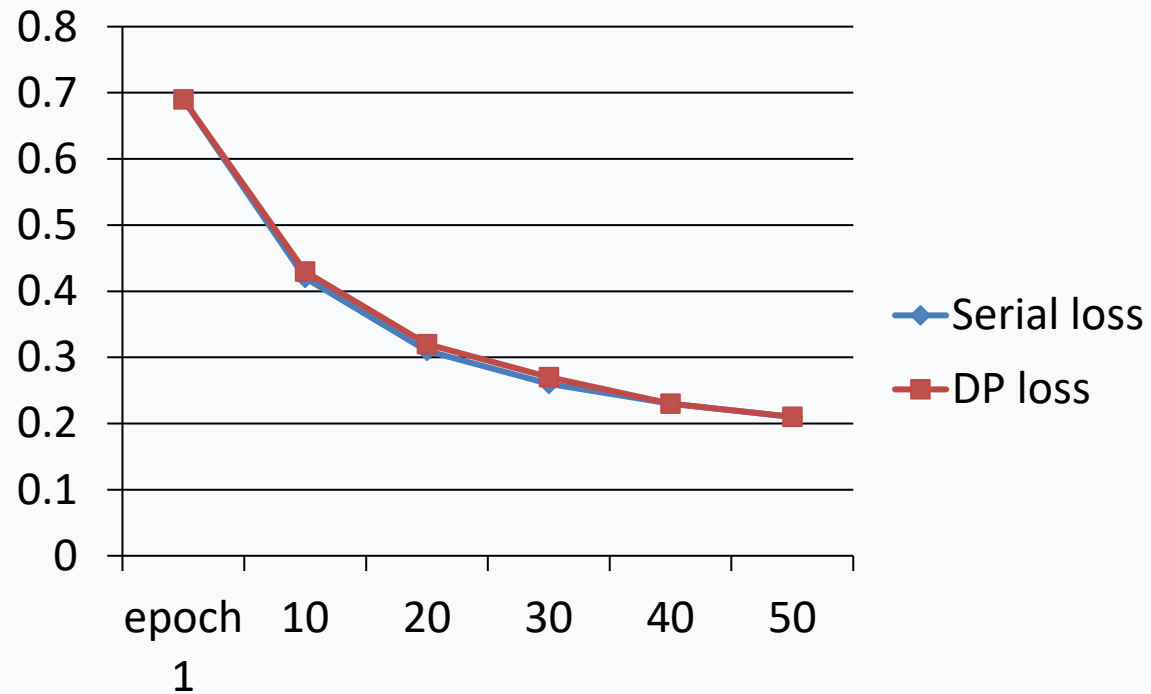
Explain

When does accuracy differ?
Where is communication
cost?

Expected Result Format

Students should report learning and systems behavior

Example loss curves



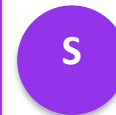
Convergence

loss decreases over epochs



Accuracy

compare final predictive quality



Systems

world size, runtime, communication count

Exit Ticket

Short reflection + training curves

Q

Answer

- Data parallelism vs model parallelism?
- Why average gradients?
- What does all-reduce do?
- Did DP match serial?
- Where could your project use this?

✓

Submit

- code/notebook
- loss/accuracy chart
- runtime table
- correctness comparison
- 1-paragraph interpretation

Instructor Notes

Teaching moves for Day 6

- 1 Connect all-reduce directly to Day 5 collectives.
- 2 Keep the first simulation framework-free with NumPy.
- 3 Distinguish gradient averaging from weight averaging.
- 4 Have students compare serial and DP loss curves.
- 5 Discuss communication/computation tradeoffs.
- 6 Preview real PyTorch DDP without requiring GPUs.

References and Further Reading

Data parallelism and PyTorch DDP

1. **PyTorch DDP tutorial:** DDP uses `torch.distributed` collectives to synchronize gradients and buffers across processes.
2. **PyTorch DDP design note:** DDP broadcasts the `state_dict` at construction, creates a reducer, and synchronizes gradients during backward.
3. **PyTorch distributed overview:** data parallelism replicates the model on every process and averages gradients before optimizer steps.
4. **`torch.distributed` docs:** `all_reduce`, `broadcast`, `gather`, `scatter`, and `reduce` are supported communication operations.